EchoEstate

Photorealistic Digital Twin Platform

Product Requirements Document

Version 1.3 | April 2026

Attribute	Detail
Project Name	EchoEstate
Type	Portfolio / Full-Stack Web Application
Timeline	3–4 Weeks (Sprint-based)
Developer Level	Junior–Intermediate Full Stack Engineer
3D Processing	Self-hosted: COLMAP + OpenSplat on RunPod Serverless GPU
Dev / Test Environment	NVIDIA A100 MIG 9 GB slice (bare-metal LXD container)
Production GPU	RunPod Serverless Endpoint — RTX 4090 (24 GB VRAM)
Deployment Target	Vercel (frontend) + Fly.io (backend) + RunPod Serverless (GPU worker)

1. Executive Summary

EchoEstate is a full-stack web application that allows real estate agents and property managers to enrich property listings with photorealistic 3D splat views — short, focused scene captures that complement photos, videos, and written descriptions. Rather than attempting to reconstruct an entire property as a single 3D walkthrough, EchoEstate treats Gaussian Splatting as one powerful lens in a multi-media listing experience.

Agents upload short 20–30 second clips of specific scenes within a property (a kitchen counter, a bay window, a bathroom vanity). Each clip is processed into a navigable 3D Gaussian Splat. These splat views are then arranged on a simple spatial canvas and surfaced to buyers alongside conventional listing media.

Version 1.3 Changes from v1.2

The GPU compute model has shifted from on-demand RunPod Pods to a RunPod Serverless Endpoint. Rather than launching and terminating a Pod per clip, the backend now submits jobs to a persistent serverless endpoint (backed by RTX 4090 workers) via the RunPod Serverless API. This simplifies infrastructure management, eliminates Pod cold-start orchestration, and provides automatic scaling across concurrent jobs. The COLMAP + OpenSplat Docker worker image is unchanged; it is now registered as the handler for the serverless endpoint.

The product model (multi-view splat gallery, View Map canvas) is unchanged from v1.2.

2. Problem Statement

2.1 The Industry Pain Point

Traditional real estate listings rely on static photography and, increasingly, 360° panoramas. Both formats suffer from fundamental limitations:

- Flat and non-interactive — users cannot freely explore a space
- Distorted — wide-angle lenses warp spatial understanding
- Expensive for high-quality virtual tours (Matterport hardware costs $3,000+)
- Lossy — fine material detail and lighting nuance are lost in compression

2.2 The Opportunity

Gaussian Splatting makes it possible to create near-photorealistic, freely navigable 3D scene captures from ordinary smartphone video. EchoEstate packages this capability as a practical listing enrichment tool — not a replacement for other media, but a powerful complement to it. A buyer browsing a listing can click a hotspot on the View Map to instantly step inside a specific scene: the kitchen island, the master bedroom corner, the garden terrace.

2.3 Why Multi-View Instead of Full Walkthrough?

Extensive feasibility testing on COLMAP + OpenSplat revealed that reconstruction quality degrades significantly with long continuous videos with fast camera motion or motion blur, open-plan spaces with insufficient visual overlap between frames, and varying lighting conditions across a long capture.

Short, focused 20–30s clips of bounded scenes avoid nearly all of these failure modes. The multi-view model also enables per-view quality gates — a bad splat can be rejected and re-captured without invalidating the rest of the listing. This makes the platform genuinely usable by non-technical agents and far more reliable as a portfolio demonstration.

2.4 Why Self-Host Instead of Using Luma AI?

Scenario	Steps	Result
Full HD 10s video — A100 9 GB MIG	3,000	~19 min (poor quality)
Full HD 10s video — A100 9 GB MIG	10,000	~1 hour (mediocre)
Full HD 10s video — A100 9 GB MIG	15,000	>3 hours (incomplete)
Projected: RunPod Serverless RTX 4090 (24 GB)	10,000	~8–12 min (target quality)
Projected: RunPod Serverless RTX 4090 (24 GB)	30,000	~25–30 min (production quality)

Self-hosting removes Luma AI API rate limit risk, eliminates per-scan cost beyond RunPod compute (~$0.37/job on RTX 4090 via serverless billing), and gives full control over step count and resolution per clip. The RunPod Serverless model additionally removes Pod lifecycle management — the endpoint auto-scales workers and the backend simply submits jobs and polls for completion.

3. Goals & Success Metrics

3.1 Project Goals

Primary Goal

Deliver a working, deployed, end-to-end application that demonstrates the full journey from scene clip upload to an interactive multi-view splat gallery arranged on a spatial View Map, within 3–4 weeks.

Secondary Goals

- Demonstrate full-stack competence across frontend, backend, GPU worker, and deployment
- Build and benchmark a COLMAP + OpenSplat Docker image registered as a RunPod Serverless endpoint handler
- Showcase frontend depth through a drag-and-drop spatial canvas (View Map) with live splat loading
- Produce clean, well-documented code suitable for a portfolio GitHub repository
- Deploy to a public URL reachable by recruiters

3.2 Success Metrics

Metric	Target	How Measured
Upload → .ply ready (production)	< 15 min per clip on RTX 4090 Serverless	RunPod Serverless job timing logs
Upload → .ply ready (dev/test)	< 45 min per clip on A100 MIG 9 GB	Manual test timing
3D viewer FPS	> 30 FPS on mid-range laptop	Browser DevTools
API error handling	100% of states covered	Manual edge case testing
View Map canvas	Drag-and-drop functional with ≥2 hotspots	Manual UX test
Mobile responsive	Listing pages usable on 375px	Chrome DevTools emulator
Deployment uptime	Public URL live for review	Recruiter walkthrough test
Cost per clip (production)	< $0.20 per scene clip via Serverless billing	RunPod billing dashboard
Code quality	README + inline comments	Peer / self review

4. User Personas

Persona 1 — Property Lister (Primary)

Name	Priya — independent real estate agent
Goal	Upload short clips of key scenes, arrange them on the View Map, and share an enriched listing with buyers
Pain point	Matterport is too expensive; phone photos don't convey space or material quality accurately
Tech comfort	Can upload files and fill in forms; not a developer

Persona 2 — Property Browser (Secondary)

Name	Rohan — prospective flat buyer relocating to a new city
Goal	Get a genuine sense of key spaces in shortlisted properties without travelling
Pain point	Photos are misleading; he has wasted weekends visiting unsuitable flats
Tech comfort	Comfortable with web apps; expects fast, intuitive navigation

5. Feature Requirements

5.1 Functional Requirements

F1 — Property Discovery (Next.js)

- Landing page with property listing grid (address, price, thumbnail, room count)
- Individual property detail page with metadata, conventional media (photos/video), and entry point to the View Map
- Basic search and filter: location keyword, min/max price, number of bedrooms
- Responsive layout — usable on desktop and mobile browsers

F2 — Scene Clip Upload & Processing Pipeline (Node.js + RunPod Serverless)

- Authenticated upload form accepting MP4 / MOV clips up to 200 MB per clip (20–30s scenes)
- Agent labels each clip with a scene name (e.g. 'Kitchen Island', 'Master Bedroom')
- Client-side upload progress indicator per clip
- Server streams clip to Cloudflare R2, then submits a job to the RunPod Serverless endpoint via the RunPod Serverless API
- The serverless endpoint handler downloads the clip from R2, runs ffmpeg → COLMAP → OpenSplat, uploads the .ply to R2, and returns the output URL
- Per-clip status lifecycle: Queued → Processing → Ready → Failed
- Failed clips surface a user-friendly error; agent can re-upload or skip that scene
- Quality gate: if COLMAP produces fewer than 30 registered images, mark clip as Failed before OpenSplat runs

F3 — View Map Canvas (Next.js + React DnD)

- After clips are processed, agent enters the View Map editor for the listing
- Canvas displays a draggable bubble per Ready view (labelled with the scene name)
- Agent positions bubbles roughly spatially (e.g. kitchen bubble top-left, bedroom bubble right)
- Optional: agent uploads a floor plan image (JPG/PNG) which becomes the canvas background
- Canvas state (bubble positions, labels, floor plan URL) saved to the property record
- Published View Map is read-only for buyers; hotspot bubbles pulse subtly to invite interaction

F4 — 3D Gaussian Splat Viewer (Svelte + Three.js)

- Clicking a hotspot bubble loads that clip's .ply file into the inline viewer panel
- Orbit controls: click-drag to rotate, scroll to zoom, right-click to pan
- Scene label and property metadata overlay visible inside viewer
- Loading state with progress bar while splat downloads
- Close / collapse viewer returns focus to the View Map

F5 — Metadata API (Go)

- REST endpoints: list properties, get property by ID, create property, update status
- Scene clip endpoints: create clip record, update clip status and .ply URL, list clips by property
- View Map endpoint: save and retrieve canvas layout (bubble positions + floor plan URL) per property
- Bookmark endpoint: save / unsave a property per user session
- Concurrent-safe — handles multiple simultaneous requests without race conditions
- Returns paginated results with cursor-based pagination

F6 — Authentication (Node.js)

- Email + password sign-up and login
- JWT-based session tokens stored in HTTP-only cookies
- Protected routes: upload, View Map editor, and bookmark features require authentication
- Basic user profile page showing uploaded properties and their clip statuses

5.2 Non-Functional Requirements

Category	Requirement
Performance	3D viewer must achieve > 30 FPS on a mid-range laptop (no dedicated GPU required)
Processing Time	Per-clip video → .ply must complete in < 15 min on RTX 4090 Serverless; < 45 min on A100 MIG (dev)

Reliability	RunPod Serverless job failures surface user-friendly errors per clip; app must not crash silently
Quality Gate	COLMAP registered image count checked before OpenSplat; clips below threshold marked Failed
Security	No API keys exposed to browser; all RunPod Serverless and R2 calls made server-side only
Scalability	Go metadata service must handle concurrent reads without data corruption; RunPod Serverless endpoint auto-scales GPU workers
Cost Control	RunPod Serverless billing is per-second of execution; idle time is not billed
Maintainability	All services have a README; environment variables documented in .env.example
Accessibility	Discovery pages meet WCAG AA contrast ratios; alt text on all images

6. Technical Architecture

6.1 Stack Overview

Layer	Technology	Responsibility
Discovery Frontend	Next.js 14 + Tailwind CSS	SSR listing pages, search, auth UI, View Map editor canvas
View Map Canvas	React DnD (or plain pointer events)	Drag-and-drop hotspot positioning, floor plan background upload
3D Viewer	Svelte + Three.js	Per-clip .ply rendering, orbit controls, metadata overlay
Upload API	Node.js / Express	File ingestion, R2 upload, RunPod Serverless job submission, polling, JWT auth
Metadata API	Go (net/http)	Property + clip CRUD, View Map layout persistence, bookmarks
GPU Worker	Docker (COLMAP + OpenSplat)	ffmpeg → COLMAP → OpenSplat per clip; registered as RunPod Serverless endpoint handler
Primary Database	PostgreSQL	Users, properties, clips, job statuses, bookmarks, View Map layouts
Cache	Redis	Session store, listing cache, RunPod job status cache
File Storage	Cloudflare R2	Raw clip uploads + processed .ply output files + floor plan images
Dev / Test GPU	NVIDIA A100 9 GB MIG (LXD)	Container development, COLMAP tuning, OpenSplat benchmarking

Production GPU	RunPod Serverless — RTX 4090 (24 GB VRAM)	Production Gaussian Splatting jobs via serverless endpoint (~$0.74/hr per active worker, billed per second)

6.2 Self-Hosted Pipeline — How It Works (Per Clip)

The per-clip pipeline runs inside a Docker container handler registered with a RunPod Serverless endpoint. Jobs are dispatched via the RunPod Serverless API and executed on RTX 4090 workers that scale automatically:

- Agent uploads a labelled 20–30s scene clip via the Upload API
- Node.js streams the clip to Cloudflare R2 and creates a clip record in PostgreSQL
- Node.js calls the RunPod Serverless API (POST /run) with the endpoint ID and a JSON input payload containing R2_CLIP_URL, CLIP_ID, and STEPS
- RunPod routes the job to an available RTX 4090 worker running the registered Docker handler image
- Handler downloads clip from R2 via signed URL
- ffmpeg extracts frames at 3–5 FPS, scaled to 960×540
- Quality gate: if fewer than 30 frames are extracted, return a Failed status in the job output
- COLMAP runs Structure from Motion: feature extraction → matching → sparse reconstruction → camera pose estimation
- Quality gate: if COLMAP registers fewer than 30 images, return a Failed status before OpenSplat runs
- OpenSplat optimises Gaussian blobs at COLMAP point cloud positions (10,000–30,000 steps depending on clip length)
- .ply file uploaded from the worker directly to Cloudflare R2; handler returns the R2 URL in its output JSON
- Node.js polling (every 30s) calls the RunPod Serverless status endpoint (GET /status/{job_id}) to check for completion, retrieves the .ply URL from the output, and updates the clip record to Ready
- View Map canvas reflects the new Ready hotspot; buyer can now click to load the splat

6.3 Database Schema (High Level)

Table	Key Columns	Notes
users	`id, email, password_hash, created_at`	Managed by Node.js auth service
properties	`id, user_id, title, address, price, bedrooms, thumbnail_url, created_at`	Core listing record
scene_clips	`id, property_id, label, status, r2_clip_url, ply_url, created_at`	status: queued | processing | ready | failed
gpu_jobs	`id, clip_id, runpod_job_id, r2_clip_url, last_polled_at, steps`	Tracks RunPod Serverless async job IDs per clip

view_map_layouts	`id, property_id, floor_plan_url, layout_json, updated_at`	layout_json: array of {clip_id, label, x, y} bubble positions
bookmarks	`user_id, property_id, created_at`	Composite primary key

6.4 View Map Layout JSON Structure

The view_map_layouts.layout_json column stores bubble positions as a JSON array:

```
{ "floor_plan_url": "https://r2.example.com/floorplan.jpg",
  "bubbles": [
    { "clip_id": "abc123", "label": "Kitchen Island", "x": 0.22, "y": 0.41 },
    { "clip_id": "def456", "label": "Master Bedroom", "x": 0.71, "y": 0.29 }
  ] }
```

x and y are normalised values (0–1) representing position as a fraction of canvas width and height. This makes the layout resolution-independent across screen sizes.

6.5 Docker Worker Image (RunPod Serverless Handler)

Layer	Contents
Base	nvidia/cuda:12.1-devel-ubuntu22.04
System deps	cmake, git, python3, ffmpeg, libboost, libsuitesparse, libceres, eigen3, libflann
COLMAP	Compiled from source with CUDA support (COLMAP 3.9+)
OpenSplat	Compiled from source with CUDA + LibTorch support
RunPod SDK	runpod Python SDK; handler.py implements the runpod.serverless.start() interface
Python	handler.py (serverless entry point), process.py, extract_frames.py (production-hardened for short clips)
Entrypoint	handler.py — receives JSON input {r2_clip_url, clip_id, steps} from RunPod; returns {ply_url} or {error} as output JSON
Output	Uploads .ply to R2, returns R2 URL in output JSON; returns error object on quality gate failure

7. API Design (Key Endpoints)

7.1 Go Metadata Service — Port 8080

Method	Endpoint	Description
GET	`/properties`	List all properties. Query params: page, limit, location, min_price, max_price, bedrooms

GET	`/properties/:id`	Get property with clip list and View Map layout
POST	`/properties`	Create property record
PATCH	`/properties/:id`	Update property metadata
GET	`/properties/:id/clips`	List all scene clips for a property with their statuses
POST	`/properties/:id/clips`	Create a scene clip record
PATCH	`/clips/:id`	Update clip status and ply_url (called by Node.js polling)
GET	`/properties/:id/viewmap`	Get View Map layout JSON for a property
PUT	`/properties/:id/viewmap`	Save / update View Map layout JSON
POST	`/bookmarks`	Save a property to user bookmarks
DELETE	`/bookmarks/:id`	Remove a bookmark
GET	`/health`	Health check for deployment monitoring

7.2 Node.js Upload Service — Port 3001

Method	Endpoint	Description
POST	`/auth/register`	Create new user account
POST	`/auth/login`	Login and receive JWT in HTTP-only cookie
POST	`/auth/logout`	Clear session cookie
POST	`/upload/clip`	Accepts multipart clip video + label, streams to R2, submits job to RunPod Serverless endpoint, creates clip record
GET	`/upload/clip/:id/status`	Returns current processing status for a clip
POST	`/upload/floorplan`	Accepts floor plan image, streams to R2, returns URL for View Map editor

8. Sprint Plan — 3 to 4 Week Timeline

The build is divided into four sprints. Sprint 0 covers Docker image development, serverless endpoint registration, and pipeline validation on the A100 MIG test environment. Sprints 1–3 build the full stack with the multi-view model and View Map canvas as the central frontend feature.

Sprint 0 (Days 1–7) — GPU Worker: Container Build, Serverless Registration & Pipeline Validation

All Sprint 0 work runs on the A100 9 GB MIG slice (free, bare-metal LXD) for local iteration, then validated on the RunPod Serverless endpoint on Day 6. No significant RunPod spend required for Days 1–5.

Day	Tasks	Deliverable
1	Baseline benchmark: run test_pipeline.sh on a 25s room clip at 960×540, 3fps, 3000 steps. Record timing and COLMAP registered image count.	Timing data logged; COLMAP quality baseline established
2	Harden extract_frames.py: resolution scaling flag, fps flag, frame count cap (max 100 frames for short clips). Validate COLMAP output quality.	extract_frames.py v2 with configurable resolution + fps
3	Implement quality gates in process.py: check frame count ≥30 before COLMAP; check registered images ≥30 before OpenSplat. Return error dict on failure.	process.py with quality gates; R2 upload via boto3; reads input JSON
4	Write Dockerfile: base nvidia/cuda:12.1, install system deps, compile COLMAP from source, compile OpenSplat from source, install runpod SDK, add handler.py implementing runpod.serverless.start(). Build and verify on MIG.	docker build succeeds; handler.py correctly wraps process.py for serverless interface
5	Run full pipeline inside Docker on MIG with a real room clip. Validate .ply in PlayCanvas viewer. Test quality gate failure path and error output JSON.	Docker container produces a viewable .ply; failure path returns correct error JSON
6	Push Docker image to Docker Hub / GHCR. Register image as RunPod Serverless endpoint. Submit one test job via RunPod Serverless API. Record actual timing at 10,000 steps.	Serverless endpoint live; test job completes; timing vs 15-min target confirmed
7	Tune step count until RTX 4090 Serverless hits <15 min for a 25s clip. Document optimal settings and endpoint ID. Sprint 0 review.	Optimal STEPS and resolution values locked; endpoint ID documented for backend config

Sprint 1 (Days 8–14) — Foundation & Backend

Day	Tasks	Deliverable
8	Repo setup, monorepo structure, Docker Compose for local Postgres + Redis, .env files	All services run locally with docker compose up
9	Go: project scaffold, database connection pool, /health endpoint, /properties CRUD, /clips CRUD	curl /properties returns empty list with 200
10	Go: View Map layout endpoints (GET + PUT), pagination, bookmark endpoints, error handling middleware	All Go endpoints tested with Postman / curl
11	Node.js: Express scaffold, JWT auth (register, login, logout), cookie middleware	Auth flow working end-to-end

12	Node.js: R2 clip upload integration, multipart upload endpoint, floor plan upload endpoint, RunPod Serverless job submission (POST /run to endpoint)	Upload endpoint streams clip to R2 and submits job to RunPod Serverless endpoint
13	Node.js: polling job (every 30s calls RunPod Serverless GET /status/{job_id}), quality gate error handling, clip status updates from output JSON	End-to-end: upload clip → Serverless processes → clip status updates in DB
14	Integration testing, fix bugs, write README for both backend services	Sprint 1 review: backend fully functional

Sprint 2 (Days 15–21) — Frontend & Viewer

Day	Tasks	Deliverable
15	Next.js scaffold, Tailwind setup, layout components (navbar, footer), design tokens	Shell of the site running at localhost:3000
16	Property listing page: fetch from Go API, grid layout, property cards with thumbnails	Listings page shows real data from database
17	Property detail page: metadata, conventional media section (photos/video), status badges for clips, entry to View Map	Detail page complete with all states
18	Auth UI: login and register pages wired to Node.js, protected route HOC	Full auth flow in the browser
19	Upload page: multi-clip upload form (label + file per clip), per-clip progress bars, per-clip status polling	Agent can upload multiple clips and see live per-clip status
20	View Map editor: drag-and-drop canvas with bubble positioning, optional floor plan image upload as background, save layout to Go API	Agent can arrange hotspot bubbles spatially; layout persists
21	Svelte 3D viewer: Three.js GSplat loader, orbit controls, metadata overlay; wire to View Map hotspot click; loading skeleton; mobile responsiveness pass	Sprint 2 review: clicking a hotspot loads the splat inline

Sprint 3 (Days 22–28) — Polish, Testing & Deployment

Day	Tasks	Deliverable
22	Deploy Go + Node.js to Fly.io, configure environment variables and secrets including RUNPOD_ENDPOINT_ID and RUNPOD_API_KEY	Both backend services live on fly.io subdomains
23	Deploy Next.js to Vercel, configure API_URL environment variables pointing to Fly.io	Full application live on public URL
24	End-to-end test on deployed environment: upload 3 clips for one property, arrange View Map, view splats in browser	Confirmed working on production

25	UI polish: empty states, loading skeletons, error toasts, hover states, typography refinement	Application feels complete and professional
26	Seed 3 demo properties using 2–4 pre-made .ply files each (own captures or community captures)	Recruiter can explore View Maps and splat tours immediately
27	Write project README: architecture diagram, View Map explanation, pipeline docs, Docker image + serverless endpoint docs, screenshots	GitHub repository is portfolio-ready
28	Buffer: remaining bugs, record a 3-minute demo video for portfolio	Project complete and submitted

9. Out of Scope (for this sprint)

- Real payment integration (Stripe, Razorpay)
- Agent analytics dashboard
- Native mobile app — responsive web covers mobile
- Real-time notifications — polling on the upload page is sufficient
- Multi-language / i18n support
- Automated testing suite — manual testing used within the sprint
- Webhook-based RunPod Serverless completion callbacks — polling is sufficient for V1
- Auto-scaling GPU worker pool management — RunPod Serverless handles scaling automatically
- Automatic floor plan generation from clips — V2 enhancement
- Stitching multiple clips into a single unified 3D scene — intentionally out of scope

10. Risk Register

Risk	Likelihood	Impact	Mitigation
COLMAP fails on fast / blurry clip	Medium	Low	Enforce 3fps, 960×540 input; validate frame count and registered image count before OpenSplat
OpenSplat OOM on MIG 9 GB during dev	High	Medium	Use 480×270 during dev; full HD only on RunPod Serverless RTX 4090
RunPod Serverless cold start time	Medium	Medium	Pre-pull Docker image to RunPod network volume; measure cold start in Sprint 0 Day 6; endpoint keeps workers warm if configured
RunPod Serverless API changes or downtime	Low	High	Pre-generate demo .ply files as fallback; monitor RunPod status page
View Map canvas UX complexity	Medium	Medium	Build with pointer events first; integrate react-dnd only if native events are insufficient
Three.js GSplat loader performance	Medium	High	Test with small .ply files first; fall back to @luma-ai/luma-web embed if needed

Go + Next.js CORS configuration	High	Medium	Set permissive CORS in development; tighten in Sprint 3
Fly.io deployment configuration time	Medium	Medium	Allocate full Day 22 to deployment; use Railway as backup PaaS
Sprint 0 Docker build time	High	Medium	Cache compiled binaries in Docker layer; build once, push to registry; reuse for serverless endpoint

11. How to Discuss This Project in an Interview

When asked: What is the project?

EchoEstate is a full-stack real estate platform that enriches property listings with photorealistic 3D scene captures. Agents upload short 20–30 second clips of key spaces — a kitchen island, a bathroom vanity, a bay window — and each clip is processed into a navigable Gaussian Splat view. These views are then arranged on a drag-and-drop View Map canvas, so buyers can click a hotspot and step inside any scene. I built the entire stack: a Go metadata service, a Node.js upload and RunPod Serverless dispatch pipeline, a Next.js discovery frontend with the View Map editor, a Svelte-based 3D viewer using Three.js, and a Docker GPU worker registered as a RunPod Serverless endpoint handler running COLMAP and OpenSplat on RTX 4090 instances.

When asked: Why per-scene clips instead of a full walkthrough?

This was a deliberate product and engineering decision based on testing. COLMAP reconstruction degrades badly with long continuous videos — motion blur, insufficient frame overlap, and varying lighting all cause failures. Short focused clips of bounded scenes avoid nearly all of these failure modes. It also gives me a per-view quality gate: if one clip produces a bad splat, the agent re-records just that scene rather than the entire property. And from a UX perspective, a curated view gallery is actually more honest than promising a full 3D walkthrough — buyers get photorealistic immersion for the specific scenes that matter most.

When asked: Tell me about the View Map — that sounds complex.

The View Map is the centrepiece of the frontend and the part I'm most proud of technically. It's a spatial canvas built in Next.js where each processed splat view is represented as a draggable hotspot bubble. Agents position the bubbles roughly where each scene sits in the property — kitchen top-left, bedroom top-right, and so on. Optionally they can upload a floor plan image as the canvas background for precision. Bubble positions are stored as normalised x/y values (0–1) so the layout is resolution-independent. On the buyer side the canvas is read-only and clicking a bubble loads that view's .ply file into the Svelte viewer inline. The drag-and-drop was built with pointer events first and React DnD for polish — it was a great exercise in managing complex UI state.

When asked: Why RunPod Serverless instead of managing Pods?

I chose RunPod Serverless because it removes Pod lifecycle complexity from the application layer. Instead of launching a Pod, waiting for it to start, running the job, and then terminating it — all of which requires careful error handling and cold-start management — I simply POST a job payload to my registered endpoint and poll for the result. RunPod handles worker scaling, cold starts, and termination automatically. Billing is per second of actual execution rather than per Pod-hour, which makes cost estimation very predictable per clip. And because the endpoint runs the same Docker image I developed locally, the local–production parity is excellent.

When asked: What would you do differently in V2?

Replace the polling loop with a RunPod Serverless webhook for real-time clip completion. Add a job queue like BullMQ for retry logic and failure handling. Explore converting .ply output to the more compact .splat format for faster viewer load times. Longer term, look at auto-generating a schematic floor plan from the clip metadata — using COLMAP camera poses to infer rough spatial relationships between scenes and pre-position the bubbles automatically.

12. Appendix — Useful Resources

Resource	URL / Notes
RunPod Serverless Docs	docs.runpod.io/serverless — endpoint creation, job submission API, status polling, handler SDK
RunPod Python SDK	github.com/runpod/runpod-python — runpod.serverless.start() for handler.py
OpenSplat	github.com/pierotofy/OpenSplat — open-source Gaussian Splatting trainer
COLMAP	colmap.github.io — Structure from Motion and MVS library
antimatter15/splat	github.com/antimatter15/splat — lightweight Three.js GSplat / .ply loader
PlayCanvas .ply Viewer	playcanvas.com/viewer — drag-and-drop .ply validation during development
Cloudflare R2 Docs	developers.cloudflare.com/r2 — S3-compatible object storage with zero egress fees
React DnD	react-dnd.github.io — drag-and-drop primitives for the View Map canvas
Fly.io deployment	fly.io/docs — deploy Go and Node.js as separate Fly apps
Vercel deployment	vercel.com/docs/frameworks/nextjs — one-click Next.js deploy
3DGS original paper	arxiv.org/abs/2308.04079 — the 2023 Gaussian Splatting paper (for interview depth)
Pre-made .ply files	lumalabs.ai/captures — free community captures for demo seeding